

Lab Title:- HTTP ANALYSIS USING WIRESHARK - EMBEDDED IMAGE TRAFFIC

Student Name:- BADAMASI MUNTAKA MUHAMMAD

Student ID:- 2025FWSD11563

Course Name:- BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

Instructor Name:- COMMANDANT AMINU IDRIS

Date Of Submission:- 10/09/2026

Executive summary

The lab covers a hands on practical on how to use wireshark to examine http packet traffic and also use it to recover image that are embedded inside the http packet traffic.

Lab Objectives

The objectives of the lab is to understand how to use wireshark to conduct some analysis on embedded image contain in http packet traffic

Tools and Resources Used

- Kali linux:- A penetration testing platform used for security assessments.

Kali linux tools

- ❖ Apache2:- this is the Apache HyperText Transfer Protocol (HTTP) server program that is designed to be run as a standalone daemon process.
 - ❖ tshark:- Dump and analyze network traffic
 - ❖ wireshark:- is a GUI network protocol analyzer that lets you interactively browse packet data from a live network or from a previously saved capture file.
 - ❖ web browser(Mozilla browser):- a web browser for browsing/searching web pages.
 - ❖ sha256sum:- Print/compute or check SHA256 (256-bit) checksums message digest.
 - ❖ curl:- a tool for transferring data from or to a server using URLs.
 - ❖ File:- determine file type
 - ❖ Identify:- describes the format and characteristics of one or more image files.
-

Methology

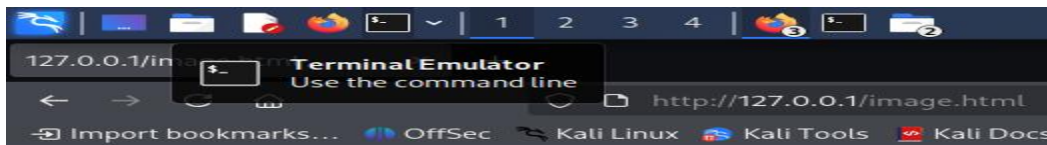
- I launched the kali linux terminal.
- I navigate to the html folder.

- I created a file(LAB2) in the html folder.
- I launched the Mozilla firefox browser.
- I search for the webpage created in the html folder(http://127.0.0.1/LAB2.html).
- I launched the wireshark to capture the LAB2.html traffic from the Mozilla firefox browser.
- I analyse the LAB2.html traffic and extract the embedded image from the traffic.
- I save the embedded image inside a folder(RECOVERED_IMAGE) then compute it sha256 hash value and the compare the hash with the original image hash value

Screenshot and Evidence

USING TSHARK

SCREENSHOT SHOWING THE WEBPAGE WITH EMBEDDED IMAGE



SBT-DF203 Image Traffic

Analyst: MUNTAKA BADAMASI MUHAMMAD



SCREENSHOT SHOWING EMBEDDED IMAGE INFORMATION & HASH VALUE

```
(kali@kali)-[~]
$ cd Desktop

(kali@kali)-[~/Desktop]
$ ls
building_20201108_221645.jpg  IMG_20250920_145054_818.jpg  mmm  Nessus-10.12.4-debian10_amd64.deb  rockyou.txt  usb_image.001
Ch01InChap01.dd             mmn.txt                     mon.txt  output                               usb_image    USB-image.001

(kali@kali)-[~/Desktop]
$ file building_20201108_221645.jpg
building_20201108_221645.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 640x415, components 3

(kali@kali)-[~/Desktop]
$ sha256sum building_20201108_221645.jpg
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb  building_20201108_221645.jpg

(kali@kali)-[~/Desktop]
$
```

SCREENSHOT SHOWING THE CAPTURED BROWSER REQUEST

```
(root@kali)~[~/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
Running as user "root" and group "root". This could be dangerous.
  1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 41182 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=26
28756316 TSecr=0 WS=512
  2 0.000019270 127.0.0.1 → 127.0.0.1 TCP 74 80 → 41182 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PE
RM TSval=2628756316 TSecr=2628756316 WS=512
  3 0.000032323 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2628756316 TSecr
=2628756316
  4 0.007644802 127.0.0.1 → 127.0.0.1 HTTP 529 GET /image.html HTTP/1.1
  5 0.008085154 127.0.0.1 → 127.0.0.1 TCP 66 80 → 41182 [ACK] Seq=1 Ack=464 Win=65024 Len=0 TSval=2628756324 TSr
cr=2628756324
  6 0.008846653 127.0.0.1 → 127.0.0.1 HTTP 597 HTTP/1.1 200 OK (text/html)
  7 0.008982646 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=464 Ack=532 Win=65024 Len=0 TSval=2628756325 T
Secr=2628756325
  8 0.128416333 127.0.0.1 → 127.0.0.1 HTTP 544 GET /building_20201108_221645.jpg HTTP/1.1
  9 0.128623789 127.0.0.1 → 127.0.0.1 TCP 32834 HTTP/1.1 200 OK
 10 0.128632725 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=942 Ack=33300 Win=94720 Len=0 TSval=2628756445
TSecr=2628756445
 11 0.128652383 127.0.0.1 → 127.0.0.1 TCP 32834 80 → 41182 [PSH, ACK] Seq=33300 Ack=942 Win=65536 Len=32768 TSva
l=2628756445 TSecr=2628756444
 12 0.128659788 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=942 Ack=66068 Win=61952 Len=0 TSval=2628756445
TSecr=2628756445
 13 0.128673305 127.0.0.1 → 127.0.0.1 HTTP 3480 HTTP/1.1 200 OK (JPEG JFIF image)
 14 0.128869240 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=942 Ack=69482 Win=94720 Len=0 TSval=2628756445
TSecr=2628756445
 15 0.284361042 127.0.0.1 → 127.0.0.1 HTTP 524 GET /favicon.ico HTTP/1.1
 16 0.284536650 127.0.0.1 → 127.0.0.1 HTTP 593 HTTP/1.1 404 Not Found (text/html)
 17 0.284545405 127.0.0.1 → 127.0.0.1 TCP 66 41182 → 80 [ACK] Seq=1400 Ack=70009 Win=125952 Len=0 TSval=26287566
00 TSecr=2628756600
00 TSecr=2628756600

(root@kali)~[~/SBT-DF203-Lab2]
```

SCREENSHOT SHOWING HTTP REQUEST LIST SHOWING HTML & IMAGE GETS

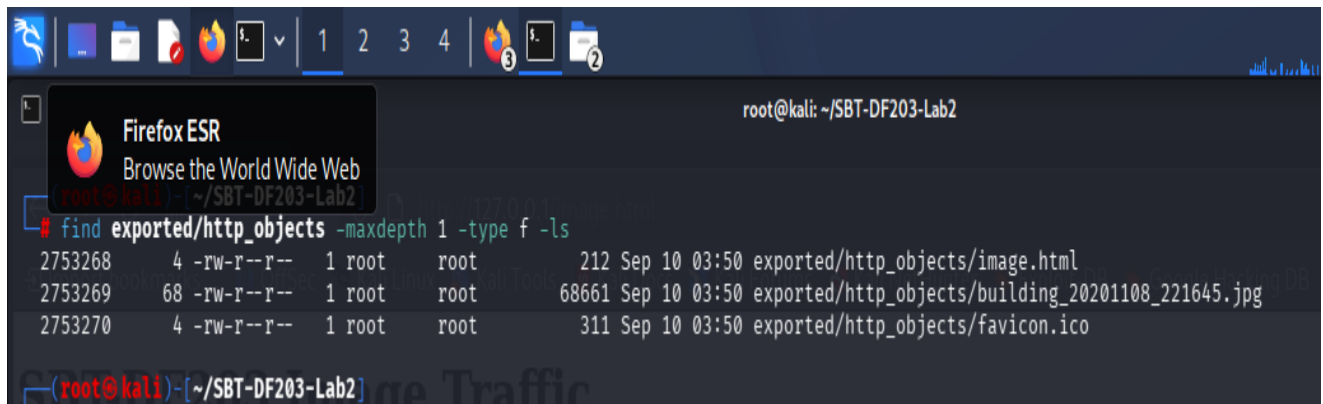
```
(root@kali)~[~/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -
e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host \
| tee reports/http_object_requests.tsv
Running as user "root" and group "root". This could be dangerous.
 4      2026-09-10T03:39:22.420474816-0400      0      127.0.0.1      41182      12
7.0.0.1 80      GET      /image.html      127.0.0.1
 8      2026-09-10T03:39:22.541246347-0400      0      127.0.0.1      41182      12
7.0.0.1 80      GET      /building_20201108_221645.jpg      127.0.0.1
15      2026-09-10T03:39:22.697191056-0400      0      127.0.0.1      41182      12
7.0.0.1 80      GET      /favicon.ico      127.0.0.1

(root@kali)~[~/SBT-DF203-Lab2]
```

SCREENSHOT SHOWING IMAGE RESPONSE HEADERS

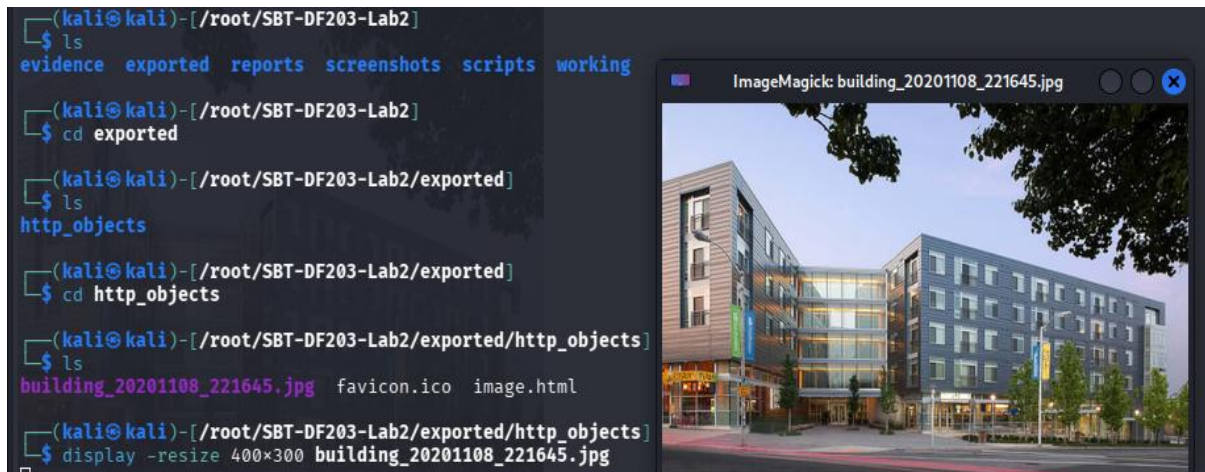
```
(root@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.conten
t_type -e http.content_length \
  | tee reports/http_object_responses.tsv
Running as user "root" and group "root". This could be dangerous.
6      2026-09-10T03:39:22.421676667-0400      0      200      text/html      19
5
13     2026-09-10T03:39:22.541503319-0400      0      200      image/jpeg     68
661
16     2026-09-10T03:39:22.697366664-0400      0      404      text/html; charset
=iso-8859-1      311
```

SCREENSHOT SHOWING THE EXPORT OBJECT WINDOW



```
(root@kali)-[~/SBT-DF203-Lab2]
$ find exported/http_objects -maxdepth 1 -type f -ls
2753268    4 -rw-r--r--    1 root    root        212 Sep 10 03:50 exported/http_objects/image.html
2753269   68 -rw-r--r--    1 root    root       68661 Sep 10 03:50 exported/http_objects/building_20201108_221645.jpg
2753270    4 -rw-r--r--    1 root    root        311 Sep 10 03:50 exported/http_objects/favicon.ico
```

SCREENSHOT SHOWING THE RECOVERED IMAGE



```
(kali@kali)-[/root/SBT-DF203-Lab2]
$ ls
evidence  exported  reports  screenshots  scripts  working

(kali@kali)-[/root/SBT-DF203-Lab2]
$ cd exported

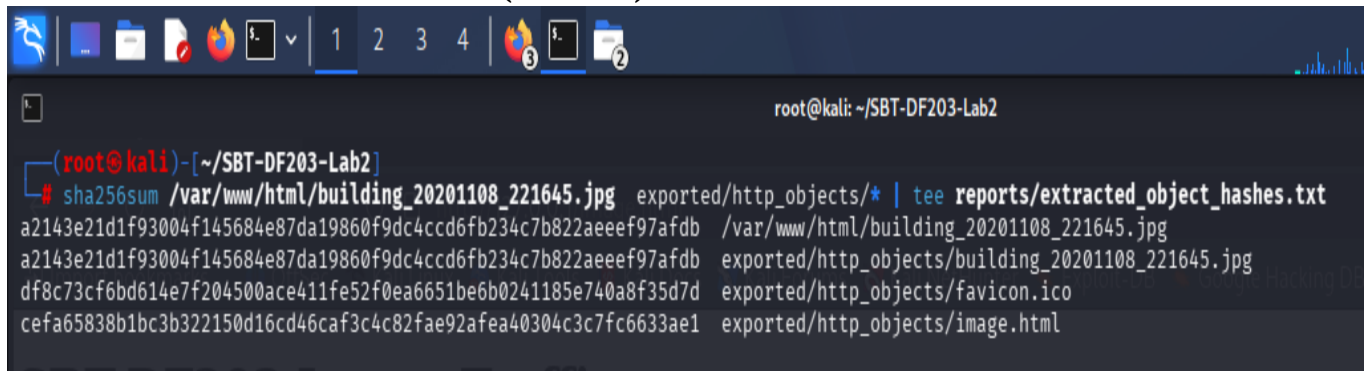
(kali@kali)-[/root/SBT-DF203-Lab2/exported]
$ ls
http_objects

(kali@kali)-[/root/SBT-DF203-Lab2/exported]
$ cd http_objects

(kali@kali)-[/root/SBT-DF203-Lab2/exported/http_objects]
$ ls
building_20201108_221645.jpg  favicon.ico  image.html

(kali@kali)-[/root/SBT-DF203-Lab2/exported/http_objects]
$ display -resize 400x300 building_20201108_221645.jpg
```

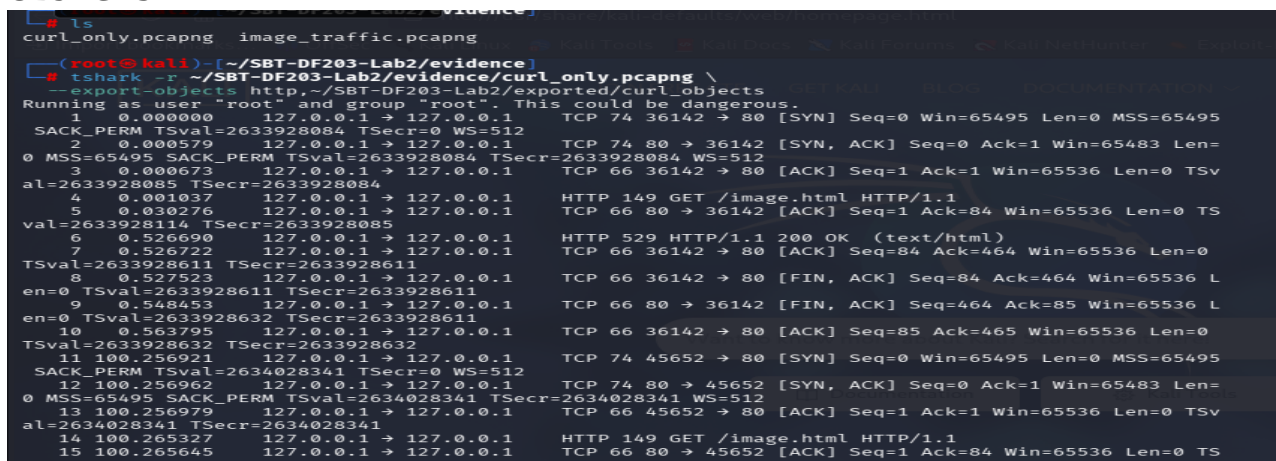
SCREENSHOT SHOWING HASH (SHA 256) COMPARISON



```
root@kali: ~/SBT-DF203-Lab2

(root@kali)~[~/SBT-DF203-Lab2]
# sha256sum /var/www/html/building_20201108_221645.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb /var/www/html/building_20201108_221645.jpg
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb exported/http_objects/building_20201108_221645.jpg
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon.ico
cefa65838b1bc322150d16cd46caf3c4c82fae92afea40304c3c7fc6633ae1 exported/http_objects/image.html
```

SCREENSHOT SHOWING CURL-ONLY CAPTURED SHOWING REQUESTED OBJECTS



```
(root@kali)~[~/SBT-DF203-Lab2/evidence]
# curl_only.pcapng image_traffic.pcapng
--export-objects http://~/SBT-DF203-Lab2/exported/curl_objects
Running as user "root" and group "root". This could be dangerous.
1 0.000000 127.0.0.1 → 127.0.0.1 TCP 74 36142 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
SACK_PERM TSval=2633928084 TSecr=0 WS=512
2 0.000579 127.0.0.1 → 127.0.0.1 TCP 74 80 → 36142 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=
0 MSS=65495 SACK_PERM TSval=2633928084 TSecr=2633928084 WS=512
3 0.000673 127.0.0.1 → 127.0.0.1 TCP 66 36142 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSv
al=2633928085 TSecr=2633928084
4 0.001037 127.0.0.1 → 127.0.0.1 HTTP 149 GET /image.html HTTP/1.1
5 0.030276 127.0.0.1 → 127.0.0.1 TCP 66 80 → 36142 [ACK] Seq=1 Ack=84 Win=65536 Len=0 TS
val=2633928114 TSecr=2633928085
6 0.526690 127.0.0.1 → 127.0.0.1 HTTP 529 HTTP/1.1 200 OK (text/html)
7 0.526722 127.0.0.1 → 127.0.0.1 TCP 66 36142 → 80 [ACK] Seq=84 Ack=464 Win=65536 Len=0
TSval=2633928611 TSecr=2633928611
8 0.527523 127.0.0.1 → 127.0.0.1 TCP 66 36142 → 80 [FIN, ACK] Seq=84 Ack=464 Win=65536 L
en=0 TSval=2633928611 TSecr=2633928611
9 0.548453 127.0.0.1 → 127.0.0.1 TCP 66 80 → 36142 [FIN, ACK] Seq=464 Ack=85 Win=65536 L
en=0 TSval=2633928632 TSecr=2633928611
10 0.563795 127.0.0.1 → 127.0.0.1 TCP 66 36142 → 80 [ACK] Seq=85 Ack=465 Win=65536 Len=0
TSval=2633928632 TSecr=2633928632
11 100.256921 127.0.0.1 → 127.0.0.1 TCP 74 45652 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
SACK_PERM TSval=2634028341 TSecr=0 WS=512
12 100.256962 127.0.0.1 → 127.0.0.1 TCP 74 80 → 45652 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=
0 MSS=65495 SACK_PERM TSval=2634028341 TSecr=2634028341 WS=512
13 100.256979 127.0.0.1 → 127.0.0.1 TCP 66 45652 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSv
al=2634028341 TSecr=2634028341
14 100.265327 127.0.0.1 → 127.0.0.1 HTTP 149 GET /image.html HTTP/1.1
15 100.265645 127.0.0.1 → 127.0.0.1 TCP 66 80 → 45652 [ACK] Seq=1 Ack=84 Win=65536 Len=0 TS
```

USING WIRESHARK

SCREENSHOT SHOWING THE WEBPAGE WITH EMBEDDED IMAGE



SCREENSHOT SHOWING EMBEDDED IMAGE INFORMATION & HASH VALUE

```
(kali@kali)~$ cd Desktop
(kali@kali)~/Desktop$ ls
building_20201108_221645.jpg  IMG_20250920_145054_818.jpg  mm  Nessus-10.12.4-debian10_amd64.deb  rockyou.txt  usb_image.001
Ch01InChap01.dd             mmn.txt                      mon.txt  output  usb_image  USB-image.001

(kali@kali)~/Desktop$ file building_20201108_221645.jpg
building_20201108_221645.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 640x415, components 3

(kali@kali)~/Desktop$ sha256sum building_20201108_221645.jpg
a2143e21d1f93004f145684e87da19850f9dc4ccd6fb234c7b822aeeef97afdb  building_20201108_221645.jpg
```

SCREENSHOT SHOWING THE CAPTURED BROWSER REQUEST

Firefox ESR *Loopback: lo

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	54742 → 80 [SYN] Seq=0 Win=65535
2	0.000020927	127.0.0.1	127.0.0.1	TCP	74	80 → 54742 [SYN, ACK] Seq=0 Ack=54742
3	0.000035409	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=1 Ack=1
4	0.010677408	127.0.0.1	127.0.0.1	HTTP	529	GET /image.html HTTP/1.1
5	0.010718068	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [ACK] Seq=1 Ack=40
6	0.011263857	127.0.0.1	127.0.0.1	HTTP	597	HTTP/1.1 200 OK (text/html)
7	0.011270998	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=464 Ack=5
8	0.188209173	127.0.0.1	127.0.0.1	HTTP	544	GET /building_20201108_221645.jpg
9	0.188435025	127.0.0.1	127.0.0.1	TCP	32834	80 → 54742 [ACK] Seq=532 Ack=5
10	0.188444765	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=942 Ack=5
11	0.188464899	127.0.0.1	127.0.0.1	TCP	32834	80 → 54742 [PSH, ACK] Seq=333 Ack=5
12	0.188467789	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=942 Ack=5
13	0.188479423	127.0.0.1	127.0.0.1	HTTP	3480	HTTP/1.1 200 OK (JPEG JFIF 3)
14	0.188698482	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=942 Ack=5
15	0.327736150	127.0.0.1	127.0.0.1	HTTP	524	GET /favicon.ico HTTP/1.1
16	0.329840607	127.0.0.1	127.0.0.1	HTTP	593	HTTP/1.1 404 Not Found (text)
17	0.329865682	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=1400 Ack=5
18	5.336025715	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [FIN, ACK] Seq=700 Ack=5
19	5.336413213	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [FIN, ACK] Seq=1400 Ack=700
20	5.336443787	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [ACK] Seq=70010 Ack=1400

SCREENSHOT SHOWING HTTP REQUEST LIST SHOWING HTML & IMAGE GETS

This screenshot shows a Wireshark capture of an HTTP GET request. The packet list pane at the top shows four packets: a SYN, an ACK, and two TCP segments. The selected packet (No. 4) is an HTTP GET request from 127.0.0.1 to 127.0.0.1 on port 80. The packet details pane shows the request structure, including the request line, host, user-agent, accept headers, and various connection options. The packet bytes pane shows the raw data of the request.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000020927	127.0.0.1	127.0.0.1	TCP	74	80 → 54742 [SYN, ACK] Seq=1
3	0.000035409	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=1
4	0.010677408	127.0.0.1	127.0.0.1	HTTP	529	GET /image.html HTTP/1.1
5	0.010718068	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [ACK] Seq=1

Frame 4: Packet, 529 bytes on wire (4232 bits), 529 bytes captured (4232 bits) on interface 0:00:00:00:00:00 (0:00:00:00:00:00), Dst: 00:00:00:00:00:00 (0:00:00:00:00:00)

Ethernet II, Src: 00:00:00:00:00:00 (0:00:00:00:00:00), Dst: 00:00:00:00:00:00 (0:00:00:00:00:00)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 54742, Dst Port: 80, Seq: 1, Ack: 1, Len: 529

Hypertext Transfer Protocol

GET /image.html HTTP/1.1

Request Method: GET

Request URI: /image.html

Request Version: HTTP/1.1

Host: 127.0.0.1

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br, zstd

DNT: 1

Sec-GPC: 1

Connection: keep-alive

Upgrade-Insecure-Requests: 1

Sec-Fetch-Dest: document

Sec-Fetch-Mode: navigate

Sec-Fetch-Site: none

Sec-Fetch-User: ?1

SCREENSHOT SHOWING IMAGE RESPONSE HEADERS

This screenshot shows a Wireshark capture of an HTTP 200 OK response. The packet list pane at the top shows four packets: a SYN, an ACK, an HTTP GET request, and an HTTP 200 OK response. The selected packet (No. 6) is the HTTP 200 OK response from 127.0.0.1 to 127.0.0.1 on port 80. The packet details pane shows the response structure, including the status line, response phrase, date, server, last-modified, etag, accept-ranges, vary, content-type, content-length, keep-alive, and connection options. The packet bytes pane shows the raw data of the response.

No.	Time	Source	Destination	Protocol	Length	Info
3	0.000035409	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=1
4	0.010677408	127.0.0.1	127.0.0.1	HTTP	529	GET /image.html HTTP/1.1
5	0.010718068	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [ACK] Seq=1
6	0.011263857	127.0.0.1	127.0.0.1	HTTP	597	HTTP/1.1 200 OK (text/html)

Frame 6: Packet, 597 bytes on wire (4776 bits), 597 bytes captured (4776 bits) on interface 0:00:00:00:00:00 (0:00:00:00:00:00), Dst: 00:00:00:00:00:00 (0:00:00:00:00:00)

Ethernet II, Src: 00:00:00:00:00:00 (0:00:00:00:00:00), Dst: 00:00:00:00:00:00 (0:00:00:00:00:00)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 80, Dst Port: 54742, Seq: 1, Ack: 464, Len: 597

Hypertext Transfer Protocol

HTTP/1.1 200 OK

Response Version: HTTP/1.1

Status Code: 200

[Status Code Description: OK]

Response Phrase: OK

Date: Thu, 10 Sep 2026 15:12:16 GMT

Server: Apache/2.4.68 (Debian)

Last-Modified: Thu, 10 Sep 2026 07:35:17 GMT

ETag: "d4-65b1c0262404c-gzip"

Accept-Ranges: bytes

Vary: Accept-Encoding

Content-Encoding: gzip

Content-Length: 195

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

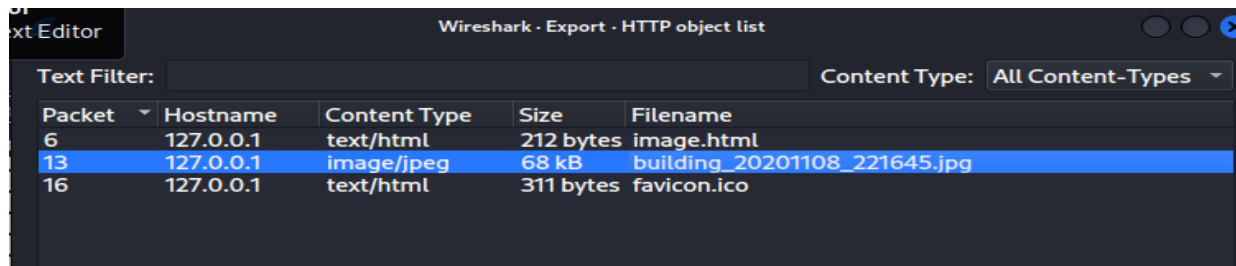
Content-Type: text/html

SCREENSHOT SHOWING RESEMBLED TCP SEGMENT LIST

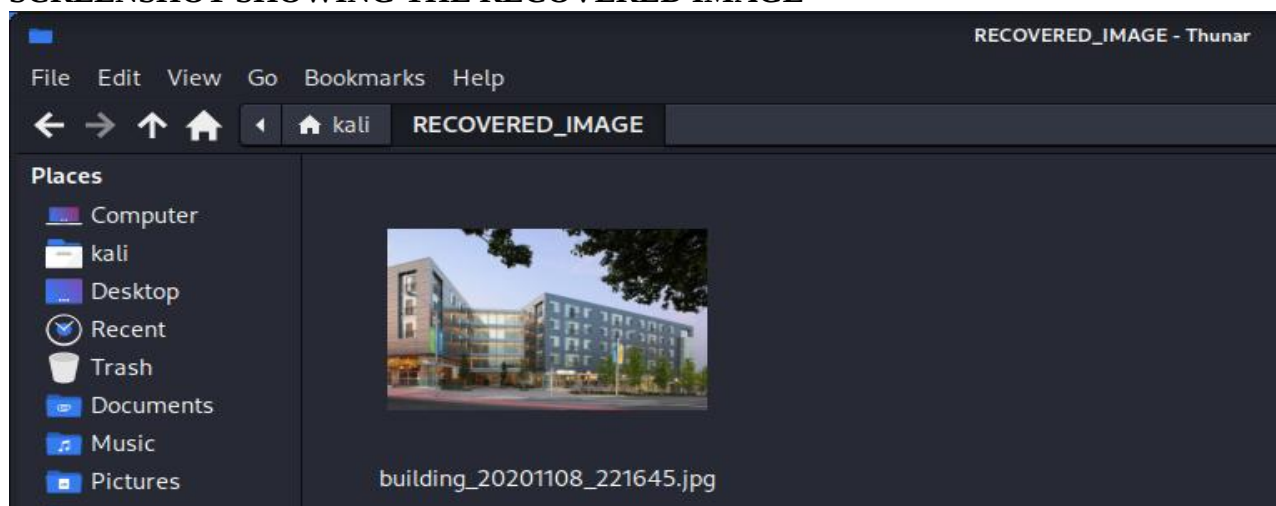
This screenshot shows a Wireshark capture of a TCP segment list. The packet list pane at the top shows 12 packets, including a SYN, an ACK, an HTTP GET request, an HTTP 200 OK response, and several other TCP segments. The selected packet (No. 1) is a SYN segment from 127.0.0.1 to 127.0.0.1 on port 80. The packet details pane shows the segment structure, including the sequence number, window size, and other options. The packet bytes pane shows the raw data of the segment.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	54742 → 80 [SYN] Seq=0 Win=65535
2	0.000020927	127.0.0.1	127.0.0.1	TCP	74	80 → 54742 [SYN, ACK] Seq=0 Win=65535
3	0.000035409	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=1 Ack=1
4	0.010677408	127.0.0.1	127.0.0.1	HTTP	529	GET /image.html HTTP/1.1
5	0.010718068	127.0.0.1	127.0.0.1	TCP	66	80 → 54742 [ACK] Seq=1 Ack=464
6	0.011263857	127.0.0.1	127.0.0.1	HTTP	597	HTTP/1.1 200 OK (text/html)
7	0.011270998	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=464 Ack=1
8	0.188209173	127.0.0.1	127.0.0.1	HTTP	544	GET /building_20201108_221645
9	0.188435025	127.0.0.1	127.0.0.1	TCP	32834	80 → 54742 [ACK] Seq=532 Ack=1
10	0.188444765	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=942 Ack=1
11	0.188464899	127.0.0.1	127.0.0.1	TCP	32834	80 → 54742 [PSH, ACK] Seq=333 Ack=1
12	0.188467789	127.0.0.1	127.0.0.1	TCP	66	54742 → 80 [ACK] Seq=942 Ack=1

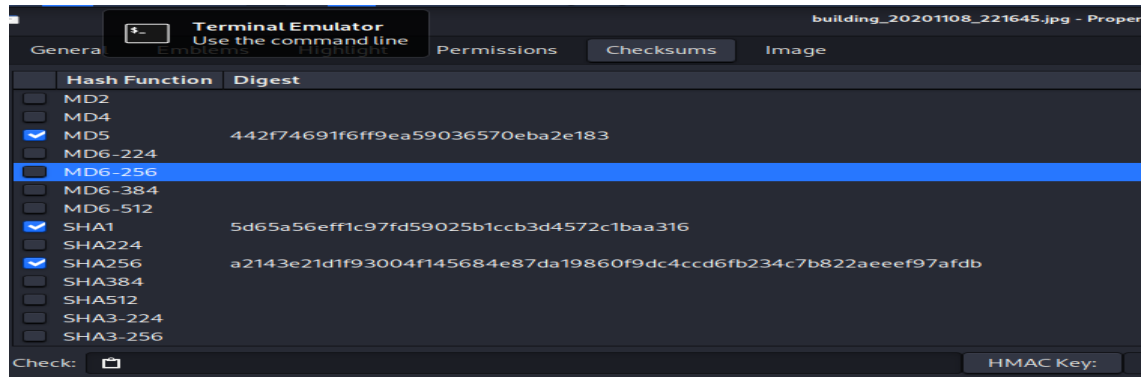
SCREENSHOT SHOWING THE EXPORT OBJECT WINDOW



SCREENSHOT SHOWING THE RECOVERED IMAGE

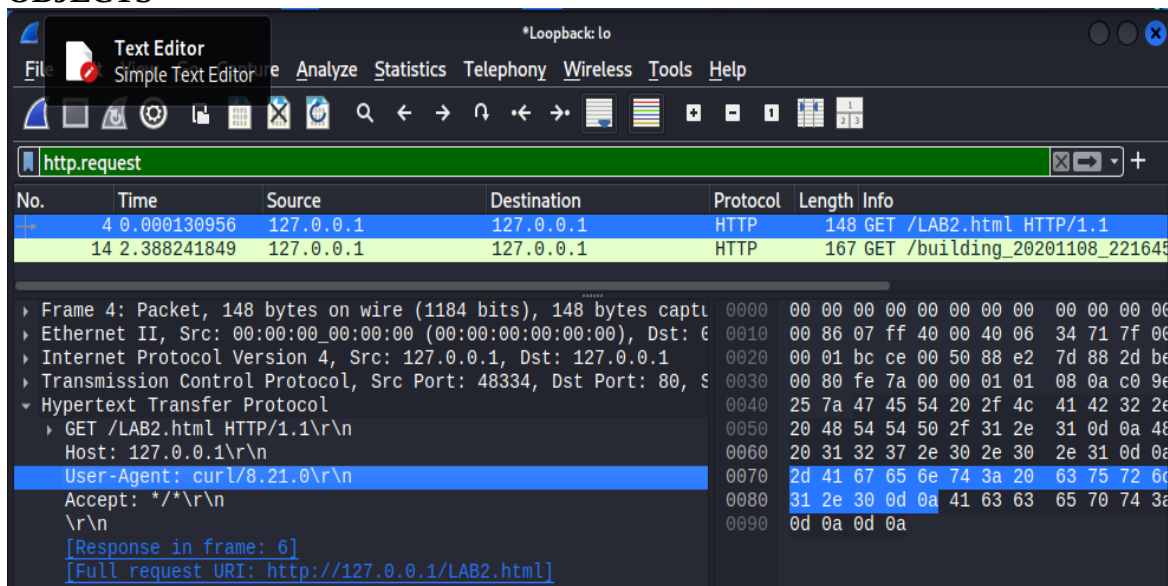


SCREENSHOT SHOWING HASH (SHA 256) COMPARISON



```
(kali@kali)-[~]
$ cd RECOVERED_IMAGE
(kali@kali)-[~/RECOVERED_IMAGE]
$ ls
building_20201108_221645.jpg
(kali@kali)-[~/RECOVERED_IMAGE]
$ sha256sum building_20201108_221645.jpg
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb  building_20201108_221645.jpg
(kali@kali)-[~/RECOVERED_IMAGE]
$
```

SCREENSHOT SHOWING CURL-ONLY CAPTURED SHOWING REQUESTED OBJECTS



Conclusion

In conclusion the lab has help in understanding some advance feature of wireshark such as using wireshark to extract an embedded image that inside a captured html page traffic.

Reference

- ICDFA course Material (SBT-DF203(basic networking skills for digital forensics)- by COMMANDANT AMINU IDRIS
 - ICDFA Recorded sessions
-

